

PRODUCT REQUIREMENT DOCUMENT (PRD)

EcoRoute: Green Logistics & Environmental Compliance Tracking System

Author: Surabhi D R	Target Stack: Java, Spring Boot 3.x, Hibernate, MySQL, Thymeleaf	Timeline: 24 Hours (Technical Interview Implementation)	Status: Approved / Production-Ready
------------------------	---	--	--

1. Executive Summary & Problem Statement

Modern commercial supply chains face massive regulatory and market pressures to measure, optimize, and report carbon footprints accurately. Organizations frequently rely on cumbersome, manual spreadsheets or hyper-complex, non-deterministic AI frameworks to track emissions. These legacy processes trigger structural implementation risk, runtime instability, and high auditing costs.

EcoRoute addresses this issue directly by providing a high-performance, deterministic enterprise logistics platform. The system computes real-time shipping carbon output, verifies compliance constraints against cargo, and generates immutable, server-rendered audit certificates. Engineered to be resilient, lightning-fast, and completely secure, EcoRoute serves as a pristine blueprint for cross-paradigm database access (JPA & JDBC) and robust backend architecture.

2. High-Level Product Vision & Goals

- **Deterministic Accuracy:** Replace unpredictable runtime logic with a robust, mathematically structured pipeline to ensure audit stability.
- **Dual-Persistence Blueprint:** Show absolute code mastery by cleanly decoupling relational transactional workloads (via Hibernate ORM) from high-volume, performance-critical analytical calculations (via optimized Spring JDBC Template).
- **Seamless Server-Side Engine:** Deliver an interface utilizing Thymeleaf and Tailwind CSS to completely avoid complex client-side application initialization under tight deadlines.
- **Deployment Readiness:** Package the platform into a production-ready infrastructure using automated CI/CD configurations, code security gating, and containerized Docker images.

3. Product Features & Detailed Requirements

3.1. Authentication & Role-Based Access Control (RBAC)

The system mandates distinct access scopes to preserve administrative data integrity and enforce operational compliance:

- **LOGISTICS_MANAGER:** Authorized to create, update, and manage shipments, evaluate transit metrics, and view tracking interfaces.
- **AUDITOR:** Granted read-only privileges across all data scopes with exclusive clearance to generate and print official Environmental Compliance Certificates.

3.2. Shipment & Carbon Management Pipeline

The core business layer calculates emissions during shipment initialization based on transport medium coefficients. When a user submits transit details, the system evaluates the carbon output via a specialized calculation engine:

Carbon Emission Logic: Emissions are derived programmatically via the following mathematical formula:

$$E = W \times D \times C_m$$

Where *E* is Total CO₂ emissions (kg), *W* represents Cargo Weight in metric tons, *D* represents Transit Distance in kilometers, and *C_m* corresponds to the constant carbon footprint factor assigned to each transportation mode.

Transport Mode	Factor (<i>C_m</i>)	Description / Operational Use-Case
AIR	0.50	High-velocity transit with elevated environmental impact limits.
SEA	0.01	Mass ocean cargo transport with optimized, ultra-low emissions.
RAIL	0.03	Highly efficient transcontinental bulk land distribution networks.
ROAD	0.12	Standard regional freight shipping and final-mile operations.

3.3. High-Performance Green Route Optimizer

To avoid object-relational mapping overhead during route resolution, the system uses direct database queries via Spring JDBC Template. This module pulls spatial coordinates from a static database matrix to calculate route distance instantly. This approach bypasses Hibernate's session level cache, showcasing advanced engineering practices for handling large, read-only analytical reference datasets.

3.4. Dynamic Server-Rendered Analytics & Certification

- **Unified Management Dashboard:** A single, high-impact overview page that displays critical system aggregates (Total Shipments Logged, Gross Metric Tons of CO₂ Emitted, Net Carbon Savings) alongside an operational tracking log.
- **Compliance Certificate Export Engine:** Generates automated, secure, print-ready HTML views styled with clean print-media borders, providing official records for environmental audits.

4. System Architecture & Technical Specifications

4.1. Complete Technical Stack Configuration

- **Core Engine Architecture:** Java 17 LTS / Spring Boot 3.x Ecosystem.
- **API Delivery Protocol:** RESTful JSON Endpoints using spring-boot-starter-web.
- **Security Standard:** Spring Security 6.x utilizing delegated BCrypt work factors.
- **Relational Storage Tier:** MySQL 8.0+ Core Enterprise Database instance.
- **Persistence Engine (Dual-Layer):** Spring Data JPA / Hibernate 6.x paired with core Spring JDBC Template.
- **View Layer Resolution:** Thymeleaf 3.x Framework styled with tailwindcss CDN engines.

4.2. Concrete MySQL Database Schema Design

The application schema explicitly isolates structured application transactions from static lookup records. The DDL script below establishes this architecture within the local MySQL engine:

```
-- Database Initialization
CREATE DATABASE IF NOT EXISTS eoroute_db;
USE ecoroute_db;

-- User Master Entities
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL,
    role VARCHAR(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Shipment Core Tracking Table (Managed by Hibernate)
CREATE TABLE shipments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    tracking_number VARCHAR(50) NOT NULL UNIQUE,
    weight_tons DOUBLE NOT NULL,
    transport_mode VARCHAR(15) NOT NULL,
    origin_city VARCHAR(100) NOT NULL,
    destination_city VARCHAR(100) NOT NULL,
    co2_emissions DOUBLE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Analytical Distance Lookups Table (Bypasses ORM via Direct JDBC Access)
CREATE TABLE distance_lookups (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    city_a VARCHAR(100) NOT NULL,
    city_b VARCHAR(100) NOT NULL,
    distance_km DOUBLE NOT NULL,
    UNIQUE KEY uq_route (city_a, city_b)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- Seed Data Insertion
INSERT INTO users (username, password, role) VALUES
('manager', '$2a$10$e0MYzXyDxAivJU3L9dMDbe9A9VzZgM4vRfZXzE1K6I2eWvW3p8Xv.',
'LOGISTICS_MANAGER'),
('auditor', '$2a$10$e0MYzXyDxAivJU3L9dMDbe9A9VzZgM4vRfZXzE1K6I2eWvW3p8Xv.', 'AUDITOR');

INSERT INTO distance_lookups (city_a, city_b, distance_km) VALUES
('Bengaluru', 'Mysuru', 145.0),
('Bengaluru', 'Mumbai', 980.5),
('Chennai', 'Bengaluru', 350.2);
```

4.3. Unified Technical Workflow Engine

The execution pipeline manages data operations through clearly separated, non-blocking execution paths:

1. The client dispatches an HTTP request, which is intercepting and validating by the **Spring Security 6.x** filters.
2. The authenticated request reaches the `ShipmentController`.
3. For route distance validation, the controller executes a high-speed direct read query via `JdbcTemplate` on the `distance_lookups` table.
4. The service layer calculates environmental output metrics and maps the payload parameters.
5. The execution flow splits: standard transactional mutations write to the `shipments` database table via ****Hibernate ORM****, while analytics and reports route to the ****Thymeleaf engine**** to instantly render the client view dashboard.

5. Security, DevOps, & Cloud Automation Requirements

5.1. Production-Grade Multi-Stage Dockerfile Containerization

To establish clean cloud isolation, the repository uses a multi-stage Docker build config. This builds the application package within a secure Maven environment and copies only the final runtime asset into an optimized JRE layer:

```
# Stage 1: Distributed Build Container Environment
FROM maven:3.8.5-openjdk-17 AS build-engine
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Hardened Runtime Environment
FROM openjdk:17-slim
WORKDIR /runtime
COPY --from=build-engine /app/target/ecoroute-0.0.1-SNAPSHOT.jar ecoroute.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "ecoroute.jar"]
```

5.2. Automated CI/CD Pipeline (GitHub Actions Workflows)

Create a workflow file located in `.github/workflows/deploy.yml` to automate verification gates and container deployments upon code check-ins:

```
name: EcoRoute Enterprise CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-verify-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Source Code Repository
        uses: actions/checkout@v3

      - name: Initialize Java Development Kit
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'maven'

      - name: Run Quality Verification & Compile Build Assets
        run: mvn clean package

      - name: Docker Container Packaging & Verification
        run: |
          docker build -t ecoroute-app:latest .
          echo "Docker image built successfully. System is deployment ready."
```

5.3. Enterprise Security & Quality Verification Profile

- **Secured Environmental Configurations:** Database credentials and operational encryption vectors are managed via environment variables (e.g., `${MYSQL_DB_PASSWORD}`) rather than plaintext configuration entries.
- **Input Validation Standards:** Request models utilize `jakarta.validation.constraints` elements (such as `@NotNull` and `@Size`) to intercept and neutralize injection vector threats before processing.

6. 24-Hour Milestones & Project Success Verification

1. **Milestone 1 (Hour 03):** Verify local MySQL connectivity and confirm successful database table creation using Hibernate's automated DLL updater.
2. **Milestone 2 (Hour 08):** Ensure all core REST API endpoints route successfully, testing request payloads and response behavior via Postman.

3. **Milestone 3 (Hour 14):** Verify that the Thymeleaf interface renders dynamic dataset views correctly and restricts access based on active user roles.
4. **Milestone 4 (Hour 20):** Confirm the Docker environment packages cleanly and verify the localized GitHub Actions workflow passes smoothly.